Dual Certificates for Agent Audit: Separating Structural Unrecoverability from Decision Relevance

Anonymous Author(s)
Affiliation
Address
email

Abstract

Auditing a deployed language-model agent requires two separable quantities: how much operative state escapes the recorded trace, and how much of that residual state drives behavior. We introduce a dual-certificate protocol. The static certificate $\varepsilon^{\mathrm{UB}}_{\mathrm{state}}$ upper-bounds residual hidden-state entropy by a min-cut on untraced channels. The dynamic certificate $\delta^{\mathrm{LB}}_{\mathrm{act}}$ lower-bounds residual decision relevance through an admissible probe taxonomy—replay, intervention, proxy—under conditional data processing. The two axes are independent. In ReAct experiments, logging ablates the static bound from $16{,}464$ to 0 bits; controlled replay separates dormant calculator from active planning tasks under the same topology as a soft policy shift (0.0163 bits, 95% CI $[0.0124, 0.0208]$) with argmax tool selections unchanged. Indexing $\delta^{\mathrm{LB}}_{\mathrm{act}}$ over hidden-channel coordinates produces an activation profile. On an LLaDA denoising trajectory, perturbations stay near the floor through early steps and rise at the final binding step (0.110 bits, 95% CI $[0.052, 0.234]$). On a multi-agent communication edge, swapping a worker's private report gives 0.901 bits, 95% CI $[0.873, 0.928]$. A Lean 4 artifact mechanizes the autoregressive zero-cut case and reduces the certificate algebra to four explicit information-theoretic axioms.

1 Introduction

Consider a ReAct-style agent shipped with an unlogged scratchpad of capacity $16{,}384$ bits per turn. A regression test on output traces shows no behavioral change, yet the same trace under controlled replay with the scratchpad disabled reveals a 0.0163-bit shift in the tool-token probability distribution on planning tasks. The disagreement is not a probing artifact. It is a structural feature of the audit: the recorded trace fails to distinguish two qualitatively different systems, one whose hidden capacity is dormant on the test distribution and one whose hidden capacity is active. Identifying which holds is the audit task.

These are two distinct quantities: how much internal state escapes the trace, and how much of that drives the next action. Output-only and black-box audit views can leave them merged into a single "hiddenness" judgment that is neither computable nor diagnostic. A system with a large logging gap but no behavioral hidden drive is indistinguishable from one with a small gap but strong drive, though the audit implications differ.

We propose a dual-certificate protocol that answers each question separately. The static certificate $\varepsilon^{\mathrm{UB}}_{\mathrm{state}}$ upper-bounds residual entropy $H(S_t \mid \tilde{T}_t)$ from deployment topology alone, using a cut-set bound on untraced channels. The dynamic certificate $\delta^{\mathrm{LB}}_{\mathrm{act}}$ lower-bounds residual decision relevance $I(S_t; A_t \mid \tilde{T}_t)$ via one of three probe classes (replay, intervention, or proxy) depending on the auditor's access regime. The two certificates are independent axes: a system with large $\varepsilon^{\mathrm{UB}}_{\mathrm{state}}$ and

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

small $\delta_{\text{act}}^{\text{LB}}$ has unrecoverable but dormant state; a system with both large has unrecoverable and active state. Both are auditable. Neither is visible to a single scalar.

The dynamic axis can also be indexed. Instead of asking only whether residual state matters, an auditor can ask *where* it matters: which module, denoising step, or communication edge carries decision relevance. This turns the certificate from a scalar diagnostic into an activation profile over the hidden-state geometry of the deployment.

Conditioning on $\tilde{T}_t$ is part of the protection, not a nuisance term. If a model writes a steganographic or compressed rationale into the recorded trace, that information is in $\tilde{T}_t$ and is conditioned away by $I(S_t; A_t \mid \tilde{T}_t)$. The dynamic certificate measures residual decision relevance beyond the logged trace; it does not count hiddenness that has already been made visible to the audit record.

Contributions. We define a dual-certificate framework $(\varepsilon_{\text{state}}^{\text{UB}}, \delta_{\text{act}}^{\text{LB}})$ for agent audit. Proposition 1 gives a topological upper bound on residual hidden-state entropy via a min-cut on untraced channels. Proposition 2 is not a new information-theoretic inequality; it is the audit reduction that turns conditional DPI into three admissible lower-bound probes—replay, intervention, and proxy—for residual decision relevance.

We evaluate the protocol on three residual-state geometries: a ReAct scratchpad module, a diffusion-LM denoising trajectory, and a multi-agent private-report edge. The same calculation is reused across these settings, but the index changes from module to time step to communication edge. Appendix calibrations cover read-only proxy estimation and a synthetic ground-truth setting where the static bound matches true hidden-state entropy. A Lean 4 artifact mechanizes Corollary 2 from Mathlib first principles and reduces Propositions 1–2 to four explicit external information-theoretic axioms (chain rule, cut-set bound, conditional Markov, conditional DPI) declared in §D.1.

2 Related Work

Probing, patching, and causal abstraction. Causal abstraction [Geiger et al., 2021], latent-knowledge elicitation [Burns et al., 2022], nullspace projection [Ravfogel et al., 2020], and amnesic probing [Elazar et al., 2021] are natural sources of proxy-style evidence: when they expose a probe variable $Z_t = \varphi(S_t)$ and the required conditional assumptions hold, its MI with action can be interpreted through the proxy certificate. Causal tracing/ROME [Meng et al., 2022], representation engineering [Zou et al., 2023], and activation addition [Turner et al., 2023] provide intervention-style evidence when the patch affects action only through S_t.

Black-box audit and network information theory. Property testing [Goldreich et al., 1998] and black-box safety auditing [Casper et al., 2024, Su et al., 2024] study what can be inferred from outputs alone. The distinction here is that output-only access can support behavioral tests, but it does not by itself provide a lower bound on δ_{act}; $\varepsilon_{\text{state}}^{\text{UB}}$ remains computable from topology when structural access is available. The static certificate proof applies a cut-set upper bound [El Gamal and Kim, 2011] to the time-unrolled DAG (full derivation in Appendix A).

Diffusion language-model agents. LLaDA instantiates large-scale masked diffusion language modeling with bidirectional denoising and instruction-following ability [Nie et al., 2025]. Recent agent work further studies diffusion LMs in multi-step decision and tool-use workflows, including matched comparisons to autoregressive agents [Zhen et al., 2026]. These systems make intermediate denoising latents a natural non-ReAct hidden channel for the dynamic certificate.

3 Setup and Audit Regime

Standard information-theoretic notation follows Cover and Thomas [2006]. All logarithms are base 2; entropy and mutual information are reported in bits.

At step t: *full trace* T_t (all intermediate activations), *recorded trace* $\tilde{T}_t \subseteq T_t$ (auditor-visible), *operative state* S_t (internal computation), *action* A_t (next token), *unrecorded sources* U_t (exogenous inputs not in $\tilde{T}_t$).

3.1 Two Core Audit Quantities

Definition 1 (Core Audit Quantities)**.** *The two quantities targeted by the dual certificate are:*

- ***Structural unrecoverability:*** $\varepsilon_{state} := H(S_t \mid \tilde{T}_t)$*, the residual entropy of the operative state given the visible trace.*
- ***Residual decision relevance:*** $\delta_{act} := I(S_t; A_t \mid \tilde{T}_t)$*, the mutual information between operative state and next action given the visible trace.*

Neither quantity is directly computable in deployment: ε_{state} *requires recovering the full internal-state distribution, and* δ_{act} *requires observing* S_t*. The dual-certificate framework targets:*

- *a* structural upper bound $\varepsilon^{UB}_{state} \geq \varepsilon_{state}$*, computed from deployment topology (§4);*
- *an* empirical lower bound $\delta^{LB}_{act} \leq \delta_{act}$*, estimated via probe variables (§5).*

The reported audit pair is $(\varepsilon^{UB}_{state}, \delta^{LB}_{act})$*.*

Neither quantity is identifiable from behavioral observations alone: different causal graphs can produce identical joint distributions over $(\tilde{T}_t, A_t)$ pairs, so observing outputs does not fix $H(S_t \mid \tilde{T}_t)$ or $I(S_t; A_t \mid \tilde{T}_t)$. The static certificate addresses the first with structural access to topology; the dynamic certificate addresses the second with gray-box probe access.

Remark 1 (Complementarity and Inheritance)**.** *By the data processing inequality,* $\delta_{act} \leq \varepsilon_{state}$*. The certificates inherit this ordering at their respective bound directions but remain independent axes of audit evidence:*

- *If* $\varepsilon^{UB}_{state} = 0$ *(fully logged architecture), then* $\varepsilon_{state} = 0$ *and* $\delta_{act} = 0$*; any valid lower bound satisfies* $\delta^{LB}_{act} = 0$*.*
- *If* $\delta^{LB}_{act} = 0$ *under the best available probe, hidden state may still exist:* ε^{UB}_{state} *can be large while the current task does not activate the hidden capacity (dormant unrecoverable state). The contrapositive (*$\delta^{LB}_{act} > 0 \Rightarrow \varepsilon^{UB}_{state} > 0$*) follows from the DPI ordering.*

The audit interpretation is therefore two-dimensional: deployment teams must report both the residual channel budget and the strongest admissible behavioral probe, not collapse them into a single hiddenness score.

3.2 Audit Access Modes

1. *Structural access.* The auditor has the deployment architecture, logging manifest, and protocol budgets. This is sufficient to compute $\varepsilon^{\text{UB}}_{\text{state}}$ from topology (§4) but gives no dynamic lower bound.
2. *Controlled replay access.* The auditor can rerun the same system under wild and replayed states while holding the visible trace fixed. This supports replay certificates when the replay mechanism affects the action only through missing state recovery.
3. *Proxy or intervention access.* The auditor can read a proxy $Z_t = f(S_t)$ or perturb a hidden module. These support conditional-DPI lower bounds on δ_{act} (§5).

The paper's empirical core uses these access modes in a controlled ReAct agent. It does not use paraphrase- or format-based black-box replay as evidence for internal decision relevance.

4 Static Certificate: Structural Upper Bound via Untraced-Channel Capacity

The static certificate upper-bounds $\varepsilon_{\text{state}} = H(S_t \mid \tilde{T}_t)$ from deployment topology. Information not in the visible trace that can influence S_t must travel through unlogged channels. Bounding those channels' worst-case capacity bounds the hidden-state entropy.

4.1 Time-Unrolled Deployment Graph and Component Budgets

Let $G_t = (V_t, E_t)$ be the time-unrolled DAG of the deployment up to step t. Nodes are state updates, tool calls, memory reads/writes, messages; edges are information channels. Three object classes: unrecorded sources U_t (retrieval results, external memory, unlogged messages), the visible trace $\tilde{T}_t$, and the operative state S_t. Let $\mathcal{C}_{\text{unlogged}}$ be all reachable edge cuts separating U_t from S_t.

A set of edges is *software-orthogonal* if, conditioned on $\tilde{T}_t$, the channel outputs factor: $p(\{y_e\}_{e\in E'} \mid \{x_e\}, \tilde{T}_t) = \prod_{e\in E'} p(y_e \mid x_e, \tilde{T}_t)$. This holds when unlogged channels use independent API calls or separate memory regions (see Appendix A for non-orthogonal remediation).

Lemma 1 (Additive decomposition)**.** *Under software orthogonality with per-edge budgets c_e, the induced cut capacity decomposes: $C_{cut}(\Omega) \le \sum_{e\in E(\Omega,\Omega^c)} c_e$.*

When software orthogonality fails (for example, when two unlogged channels share a hidden state variable) the per-edge sum $\sum_{e\in E(\Omega,\Omega^c)} c_e$ still upper-bounds $C_{\text{cut}}(\Omega)$ by sub-additivity of mutual information across coupled channels: $I(X_\Omega; Y_{\Omega^c} \mid \tilde{T}_t, X_{\Omega^c}) \le \sum_e I(X_e; Y_e \mid \tilde{T}_t) \le \sum_e c_e$. Orthogonality is required for *tightness* of the additive decomposition, not for validity of the upper bound (formal treatment in Appendix A).

Per-edge budgets c_e are read from interface specifications: $K \log |\mathcal{V}|$ for a K-token text channel over vocabulary $\mathcal{V}$, $d \log Q$ for a d-dimensional quantized state, $\log |\Omega_{\text{states}}|$ for a discrete scheduler. The discretization parameter Q is an auditor-side choice (we use $Q = 256$, 8-bit audit discretization throughout); tighter or coarser discretizations shift the bound proportionally without changing the structural claim. All per-edge bounds are worst-case (maximum-entropy) bounds; tighter empirical bounds only improve the certificate.

The bound has three parts. The term $\varepsilon_{\text{nominal}}$ is the residual uncertainty that remains even after a complete internal trace; for a fully instrumented software agent this is typically zero. The cut term is an audit budget for information that can still enter S_t through unlogged interfaces. The directed-MI notation prices that worst-case flow formally, while the implemented certificate only needs per-edge budgets and a min-cut over the unlogged part of the deployment graph.

Proposition 1 (Static Certificate via Cut-Set Bound)**.** *For any cut Ω separating U_t from S_t with induced capacity $C_{cut}(\Omega) = \sup_{p(X_\Omega|\tilde{T}_t)} I(X_\Omega \to Y_{\Omega^c} \mid \tilde{T}_t, X_{\Omega^c})$,*

$$H(S_t \mid \tilde{T}_t) \;\le\; \underbrace{H(S_t \mid T_t)}_{\varepsilon_{nominal}} \;+\; \min_{\Omega} C_{cut}(\Omega).$$

Setting $\varepsilon^{UB}_{state} := \varepsilon_{nominal} + \min_\Omega C_{cut}(\Omega)$ gives $\varepsilon_{state} \le \varepsilon^{UB}_{state}$.

Here X_Ω denotes the input signals entering edges crossing the cut, Y_{Ω^c} the corresponding output signals, and X_{Ω^c} the boundary state on the sink side (formal definitions in Appendix A).

Corollary 1 (Additive form)**.** *Under software orthogonality, $\varepsilon^{UB}_{state} = \varepsilon_{nominal} + \min_{C\in\mathcal{C}_{unlogged}} \sum_{e\in C} c_e$, a discrete min-cut on the unlogged subgraph.*

Proof sketch: Chain-rule gives $H(S_t \mid \tilde{T}_t) = H(S_t \mid T_t) + I(S_t; M_t \mid \tilde{T}_t)$ where $M_t = T_t \setminus \tilde{T}_t$. Every path from U_t to S_t crosses a cut Ω, so M_t is d-separated from S_t by the cross-cut signals; DPI yields $I(S_t; M_t \mid \tilde{T}_t) \le C_{\text{cut}}(\Omega)$. Minimizing over cuts and substituting Lemma 1 completes the proof (full derivation in Appendix A).

Corollary 2 (Autoregressive zero-cut)**.** *If the system is a strict autoregressive core and $\tilde{T}_t$ includes the full context window, then $\mathcal{C}_{unlogged} = \varnothing$, every cut has zero induced capacity, and $\varepsilon^{UB}_{state} = 0$.*

5 Dynamic Certificate: Decision Relevance via Conditional DPI

The dynamic certificate targets $\delta_{\text{act}} := I(S_t; A_t \mid \tilde{T}_t)$. Since S_t is unobservable, we specify probes whose measured variables satisfy $X_t \to S_t \to A_t$ given $\tilde{T}_t$. Conditional DPI supplies the correctness argument: $I(X_t; A_t \mid \tilde{T}_t) \le \delta_{\text{act}}$.

Proposition 2 (Probe Certificates from Conditional DPI)**.** *For any admissible probe variable X_t with $X_t \to S_t \to A_t$ given $\tilde{T}_t$, the probe measurement $I(X_t; A_t \mid \tilde{T}_t)$ is a valid lower-bound certificate for δ_{act}.*

Replay certificates. Let $R_t \in \{\text{wild}, \text{replay}\}$ indicate whether a missing state fragment is reconstructed under the same visible trace $\tilde{T}_t$ (controlled re-execution, not paraphrase variation). Then $I(R_t; A_t \mid \tilde{T}_t) = \text{JS}(P_{\text{wild}}, P_{\text{replay}} \mid \tilde{T}_t) \leq \delta_{\text{act}}$. Discrete action spaces admit direct empirical JS estimation over either realized actions or model-reported action probabilities; the experiments below state which is used.

Intervention certificates. Let ξ_{hidden} be an exogenous perturbation to a suspected unlogged module (cache, memory buffer, scratchpad). If $\xi_{\text{hidden}} \to S_t \to A_t$ given $\tilde{T}_t$, then $I(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t) \leq \delta_{\text{act}}$. Discrete action spaces admit direct conditional action-distribution shift estimates, such as JS divergence; for continuous action spaces, InfoNCE [van den Oord et al., 2018] or MINE [Belghazi et al., 2018] can be used. Requires gray-box perturbation access; the most direct causal probe of the three.

Proxy certificates. Let $Z_t = f(S_t)$ be a readable coarsening of the operative state (tool-logit projection, attention summary). Since $Z_t \leftarrow S_t \to A_t$ given $\tilde{T}_t$, conditional DPI yields $I(Z_t; A_t \mid \tilde{T}_t) \leq \delta_{\text{act}}$. We estimate this quantity by fitting predictors $p(A_t \mid \tilde{T}_t)$ and $p(A_t \mid \tilde{T}_t, Z_t)$ and taking their cross-entropy gap, $\hat{I}(Z_t; A_t \mid \tilde{T}_t)$.

Aggregation. A single certificate class can miss some causal paths. Define $\delta_{\text{act}}^{\text{LB}} := \max\{I(R_t; A_t \mid \tilde{T}_t), I(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t), I(Z_t; A_t \mid \tilde{T}_t)\}$. By monotonicity of the maximum, the aggregate remains a valid lower bound on δ_{act}.

Activation profiles. The scalar certificate is the maximum over an indexed family of admissible probes. Keeping the index gives an *activation profile*

$$\Delta(j) := I(X_t^{(j)}; A_t \mid \tilde{T}_t),$$

where j may denote a scratchpad field, a denoising step, an activation layer, or a communication edge. The static certificate identifies where residual capacity can reside; the activation profile identifies where that capacity is behaviorally used. The experiments below keep this algebra fixed while changing the index: module in ReAct, time in the diffusion LM, and communication edge in the multi-agent setting.

6 Empirical Diagnostics

The experiments start with a scalar logging ablation and then ask where the dynamic signal appears. The static ablation (§6.1) uses Corollary 1 to compute the residual bit budget from topology. ReAct intervention and controlled replay (§6.2) show the same unlogged scratchpad capacity dormant on calculator tasks and active on planning tasks. The LLaDA probe in §6.3 indexes $\Delta(j)$ by denoising step; the multi-agent probe in §6.4 indexes it by private peer-report edge. Read-only proxy and synthetic ground-truth calibrations are reported in Appendices C and E. All pipelines are designed for a single M4 Max workstation.

6.1 Static Certificate: Min-Cut and Logging Ablation

We extract the time-unrolled DAG from Qwen2.5-7B-Instruct ReAct execution traces on tool-selection tasks (Fig. 1, left). Per-edge budgets c_e are assigned from interface specifications (§4); the min-cut on unlogged edges gives $\varepsilon_{\text{state}}^{\text{UB}}$. Varying logging levels recomputes the min-cut; Table 1 reports the stepwise reduction from $16{,}464$ to 0 bits. The dominant bottleneck is the scratchpad-read path ($16{,}384$ bits). The same static computation also supplies the coordinate system for the diffusion-LM and multi-agent dynamic checks that follow.

These cuts put the later dynamic probes on a common map: a scratchpad module in ReAct, a denoising interface in LLaDA, and a peer-state edge in the multi-agent system.

6.2 Dynamic Causal Certificates: Intervention and Controlled Replay

We separate the two axes with a dormant/active task split under one topology. The unlogged scratchpad is present in both cases: irrelevant for calculator tasks, necessary for planning tasks.

Logging level	$\varepsilon^{\text{UB}}_{\text{state}}$ (bits)	Bottleneck
ReAct, output only	16,464	scratchpad_read (16384) + router_logits (80)
ReAct, +log router	16,384	scratchpad_read (16384)
ReAct, +log scratchpad	0	none
ReAct, full instrumentation	0	none
LLaDA-8B-Instruct (K=128)	32,768	denoising interface
Multi-agent, all logged	0	none
Multi-agent, peer report unlogged	32,768	peer_report $\to S_t$

Table 1: Static certificate geometry. Adding ReAct logs monotonically removes residual cut capacity; the same min-cut analysis localizes capacity in a scratchpad module, denoising interface, or peer-report edge. The LLaDA bound is independent of K because repeated reuse of the same bounded latent interface does not increase the one-step information bottleneck.

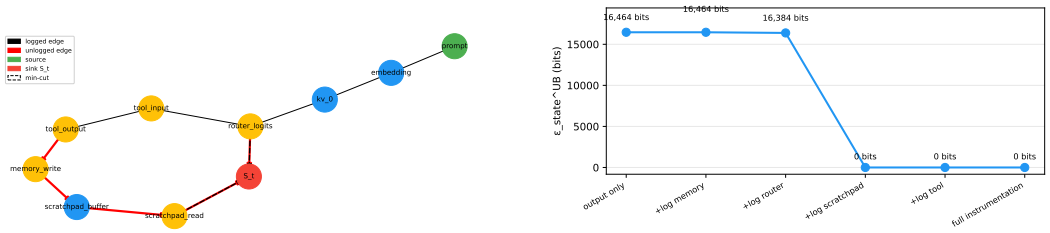

Figure 1: Left: Time-unrolled DAG extracted from Qwen2.5-7B-Instruct ReAct execution (logical edges omitted for clarity). Red edges are unlogged. Dashed edges form the min-cut. Right: Logging ablation curve. Adding instrumentation at the bottleneck drops $\varepsilon^{\text{UB}}_{\text{state}}$ stepwise to zero.

Intervention perturbs the scratchpad (masking or replacement); controlled replay removes it entirely while holding $\tilde{T}_t$ fixed. Table 2 reports JS divergence over realized tool actions. Table 3 reports JS divergence over the model's soft tool-token probability distribution; for the headline replay contrast, the argmax tool counts are unchanged, so the evidence is a policy-probability shift rather than a realized-tool flip. Both certificates are zero on dormant tasks and positive on active tasks. The topology, and therefore $\varepsilon^{\text{UB}}_{\text{state}}$, is unchanged.

The pair of tables isolates the transition from high-ε/low-δ to high-ε/high-δ: the hidden channel is structurally present in both task splits, but it becomes action-relevant only on planning tasks.

Under this fixed topology, the unlogged scratchpad contributes 16,384 bits to $\varepsilon^{\text{UB}}_{\text{state}}$ in both task splits, matching the min-cut from the extracted ReAct DAG. Intervention and replay are zero on calculator tasks and positive on planning tasks. Structural unrecoverability is set by the deployment boundary and logging policy; residual decision relevance depends on which hidden channels the task uses.

6.3 Temporal Certificate Analysis on a Diffusion LM

ReAct gives a module-level contrast: the scratchpad is either used or not used. A diffusion LM gives a temporal contrast. The hidden channel exists throughout the K-step denoising trajectory, but its decision relevance need not be constant across steps.

We use LLaDA-8B-Instruct with K=10 denoising steps on tool-selection prompts. A Gaussian perturbation (σ=5.0) is applied to layer-1 activations at steps $\{2, 4, 6, 8, 10\}$; a late-layer perturbation is tracked as a broad sensitivity control. Each point uses n=20 trajectories and a trajectory-block bootstrap with 1,000 resamples.

The pattern is late-binding. Steps 2–8 remain near the floor for the target layer (0.001–0.006 bits), while step 10 rises to 0.110 bits with 95% CI $[0.052, 0.234]$. The control layer also rises at step 10 (0.092 bits, CI $[0.043, 0.186]$), so the sensitivity is global rather than confined to the chosen layer. In the certificate geometry, LLaDA carries hidden capacity throughout denoising; it becomes action-relevant when the denoising state binds to a discrete tool choice.

Task	Perturbation	JS (bits; nats)	95% CI (bits)
Calculator (dormant)	mask 0.7	0.0000; 0.0000	$[0, 0]$
Planning (active)	mask 0.7	0.0215; 0.0149	$[0.0094, 0.0410]$
Calculator (dormant)	replace 1.0	0.0000; 0.0000	$[0, 0]$
Planning (active)	replace 1.0	0.0055; 0.0038	$[0.0016, 0.0131]$

Table 2: Dynamic intervention certificate under the same unlogged-scratchpad topology. Dormant calculator tasks show no detectable scratchpad influence; active planning tasks show positive action-distribution shift.

Task	Replay condition	JS (bits; nats)	95% CI (bits)
Calculator (dormant)	scratchpad removed	0.0000; 0.0000	$[0, 0]$
Planning (active)	scratchpad removed	0.0163; 0.0113	$[0.0124, 0.0208]$

Table 3: Controlled replay certificate over soft tool-token probability distributions. Wild episodes use the full hidden scratchpad; replay episodes use the same visible trace with the scratchpad removed. The dormant-to-active contrast mirrors the intervention result, although the headline replay contrast does not change argmax tool counts.

6.4 Multi-Agent Private-Report Intervention

The multi-agent setting changes the index again. Here the controller delegates evidence gathering to a worker, so the worker's private report is not an auxiliary scratchpad but an epistemic edge in the decision graph. The worker sees a benchmark item and writes a short report; the controller sees only that report and chooses one API from {`calculator`, `search`}. The task pool combines 200 GSM8K arithmetic items [Cobbe et al., 2021] and 200 HotpotQA multi-hop fact questions [Yang et al., 2018] with seed 42.

The main intervention replaces the worker report with a report from the opposite task class while holding the controller protocol fixed. A neutral replay removes the report content, and a same-class shuffle replaces it with another report from the same task class. An oracle-tag condition directly names and flips the API label to measure the binary ceiling. For each task and condition, the controller is sampled $k{=}8$ times; labels are parsed as {`calculator`, `search`, `invalid`}; and $\mathrm{JS}(P_{\text{base}}, P_{\text{condition}})$ is averaged with a task-block bootstrap.

The contrast is topological. In ReAct, the scratchpad matters only when the task needs it. In the multi-agent deployment, the controller's decision is routed through the private worker report, so the active site is the communication edge itself. Same-class variation is local, neutral removal is substantial, and counterfactual evidence reaches 0.901 bits, close to the oracle ceiling.

7 Discussion

7.1 One Plot: Capacity, Then Activation

The experiments read as one progression. The static certificate first maps where hidden capacity can reside: scratchpad modules, denoising interfaces, peer-state edges. The dynamic certificate then asks which of those sites affect actions. In ReAct, the same scratchpad is dormant for calculator tasks and active for planning tasks. In LLaDA, residual capacity is present throughout denoising, but action relevance concentrates at the late binding step. In the multi-agent deployment, delegation makes the private worker report the active edge.

The effect-size ordering is therefore an activation profile over coordinates, not a contradiction across probes. Under the same ReAct topology, calculator tasks leave the scratchpad dormant while planning tasks produce nonzero action-distribution shifts under intervention (0.0215 bits, CI $[0.0094, 0.0410]$) and controlled replay (0.0163 bits, CI $[0.0124, 0.0208]$). The LLaDA final denoising step rises to 0.110 bits, and the multi-agent counterfactual report edge rises to 0.901 bits. A hidden channel used as a residual supplement to an already-determined visible trace has low $\delta^{\mathrm{LB}}_{\mathrm{act}}$; a hidden channel used

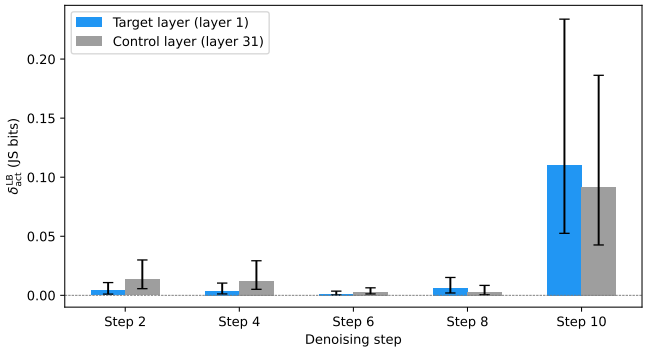

Figure 2: Temporal activation profile for LLaDA-8B-Instruct. Early denoising steps have low dynamic certificates; the final action-forming step has a much larger certificate for both target and control layers, indicating late-binding decision relevance rather than a module-local scratchpad effect.

Condition	$\delta^{\mathrm{LB}}_{\mathrm{act}}$ (JS bits)	95% CI (bits)
Neutral replay	0.358	[0.324, 0.392]
Counterfactual evidence	0.901	[0.873, 0.928]
Same-class shuffle	0.084	[0.060, 0.109]
Oracle tag upper-bound	1.000	[1.000, 1.000]

Table 4: Topological activation profile for the multi-agent deployment. Counterfactual evidence nearly saturates the binary action ceiling, while same-class shuffling remains much smaller. Hidden communication is therefore behaviorally active when the controller has delegated evidence gathering to the worker.

as the decision carrier has high $\delta^{\mathrm{LB}}_{\mathrm{act}}$. Under the protocol, a claim that an unobserved variable causally affects behavior must identify both a channel in $\varepsilon^{\mathrm{UB}}_{\mathrm{state}}$ and a probe in $\delta^{\mathrm{LB}}_{\mathrm{act}}$.

Acknowledgments

We thank the developers of Qwen2.5 and LLaDA for the open-weight models used in the ReAct and diffusion-LM experiments; the creators and maintainers of GSM8K and HotpotQA for benchmark data used in the multi-agent setting; and the Hugging Face Transformers, PyTorch, NumPy, scikit-learn, NetworkX, Matplotlib, Lean 4, Lake, and Mathlib communities for the software infrastructure supporting the experiments, figures, and formalization. Computations ran on a single Apple M4 Max workstation.

References

Rudolf Ahlswede, Ning Cai, Shuo-Yen Robert Li, and Raymond W. Yeung. Network information flow. *IEEE Transactions on Information Theory*, 46(4):1204–1216, 2000. doi: 10.1109/18.850663.

Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. Mutual information neural estimation. In *Proceedings of the 35th International Conference on Machine Learning*, pages 531–540, 2018.

Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022. doi: 10.48550/arXiv.2212.03827.

Stephen Casper, Carson Ezell, Charlotte Siegmann, Noam Kolt, Taylor L. Curtis, Benjamin Bucknall, Andreas Haupt, Kevin Wei, Jérémy Scheurer, Marius Hobbhahn, et al. Black-box access is insufficient for rigorous AI audits. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024. doi: 10.1145/3630106.3659037.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

Abbas El Gamal and Young-Han Kim. *Network Information Theory*. Cambridge University Press, 2011.

Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral consequences of information removal from a representation. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021.

Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 9574–9586, 2021.

Oded Goldreich, Shafi Goldwasser, and Dana Ron. Property testing and its connection to learning and approximation. *Journal of the ACM*, 45(4):653–750, 1998.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. doi: 10.48550/arXiv.2502.09992.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Shauli Ravfogel, Yanai Elazar, Hila Gonen, Michael Twiton, and Yoav Goldberg. Null it out: Guarding protected attributes by iterative nullspace projection. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7237–7256, 2020.

Jingtong Su, Julia Kempe, and Karen Ullrich. Mission impossible: A statistical perspective on jailbreaking LLMs. In *Advances in Neural Information Processing Systems*, 2024.

Alexander Matt Turner, Lisa Thiergart, David Udell, Gavin Leech, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.

Huiling Zhen, Weizhe Lin, Renxi Liu, Kai Han, Yiming Li, Yuchuan Tian, Hanting Chen, Xiaoguang Li, Xiaosong Li, Chen Chen, Xianzhi Yu, Mingxuan Yuan, Youliang Yan, Peifeng Qin, Jun Wang, Yu Wang, Dacheng Tao, and Yunhe Wang. DLLM agent: See farther, run faster. *arXiv preprint arXiv:2602.07451*, 2026. doi: 10.48550/arXiv.2602.07451.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

NeurIPS Paper Checklist

1. Claims Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: Yes. The abstract and introduction state the dual-certificate framework, the static upper bound (Proposition 1), the dynamic lower bound (Proposition 2), and the experimental results that correspond to them. All claims are matched by theoretical derivations (§4–5) and empirical evidence (§6).

2. Limitations Does the paper discuss limitations of the work?

Answer: Yes. Limitations are discussed in several places: the observational bottleneck and proxy-estimator bias/variance (Appendix C), the scope of claims (§7), the conservative nature of the static bound, and the dependence on software-orthogonality assumptions for tightness (§4, Appendix A).

3. Theory Assumptions and Proofs Did you state the full set of assumptions of all theoretical results, and did you include complete proofs of all theoretical results?

Answer: Yes. All assumptions are stated with each proposition (software orthogonality in Lemma 1, conditional Markov and DPI for Proposition 2). Complete proofs are given in the main text (proof sketches) and in full detail in Appendix A. A Lean 4 artifact machine-checks the structural reductions (Appendix D).

4. Experimental Result Reproducibility Did you provide a way to reproduce the experimental results?

Answer: Yes. Detailed experimental protocols are described in Appendix B, including the static-certificate computation procedure and the dynamic-certificate estimation steps (proxy, replay, intervention, and private-report intervention). The paper references implementation README files. Code, data, and the Lean artifact are included in the supplementary material.

5. Open access to data and code Did you include the code, data, and instructions needed to reproduce the main experimental results?

Answer: Yes. Code, precomputed results, and reproduction instructions are provided in the supplementary material. The experiments use the open-weight Qwen2.5-7B-Instruct model (Apache 2.0).

6. Experimental Setting/Details Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)?

Answer: Yes. Experimental details are reported in §6 and Appendix B: e.g., 5-fold cross-fitted logistic regression, trajectory-block bootstrap with 95% CIs, PCA dimensions, audit discretization $Q = 256$, perturbation levels, controller sample count k, and task-pool sampling seed. Hyperparameter choices are motivated and described.

7. Experiment Statistical Significance Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: Yes. All experimental tables report 95% confidence intervals obtained via trajectory- or task-block bootstrap (§6.2–6.4, Appendix C). Error bars are shown in the appendix proxy figure (Figure 3). The method for calculating CIs is explained in the text and Appendix B.

8. Experiments Compute Resource For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: Yes. The paper states that all pipelines ran on a single Apple M4 Max workstation (§6). This indicates the hardware class and scale; no GPU-cluster requirements are needed.

9. Code Of Ethics Have you read the NeurIPS Code of Ethics and ensured that your research conforms to it?

Answer: Yes. The authors have read and adhere to the NeurIPS Code of Ethics. The work focuses on safety-oriented auditing of AI agents and does not involve human subjects, privacy-sensitive data, or dual-use risks beyond those discussed in the paper.

10. Broader Impacts If appropriate for the scope and focus of your paper, did you discuss potential negative societal impacts of your work?

Answer: Yes. The paper develops a framework for auditing AI agents, which is inherently aimed at improving transparency and safety. Potential dual-use concerns (e.g., adversaries studying the certificates to design harder-to-audit systems) are indirectly acknowledged in the discussion of limitations and the conservative nature of the bounds (§7). The overall contribution promotes responsible AI deployment.

11. Safeguards Do you have safeguards in place for responsible release of models with a high risk for misuse (e.g., pretrained language models)?

Answer: N/A. The paper does not release any new pretrained models. It uses existing open-weight models (Qwen2.5) as-is for auditing experiments without modifying or redistributing them.

12. Licenses Did you cite the creators and respect the license and terms of use of existing assets?

Answer: Yes. The paper cites the model and dataset assets where applicable, acknowledges the supporting software communities (Qwen2.5, LLaDA, Hugging Face Transformers, PyTorch, NumPy, scikit-learn, NetworkX, Matplotlib, Lean 4, Lake, and Mathlib; §*Acknowledgments), and uses publicly available assets in accordance with their respective licenses.

13. New Assets Are you releasing new assets (datasets, code, models), and did you document them properly?

Answer: No. The experimental code, data, and Lean 4 formalization are provided in the supplementary material for review and will be publicly released and documented upon acceptance.

14. Crowdsourcing and Research with Human Subjects Did you use crowdsourcing or conduct research with human subjects?

Answer: N/A. The research does not involve crowdsourcing or human subjects.

15. IRB Approvals Did you describe potential participant risks and obtain Institutional Review Board (IRB) approvals (or an equivalent approval/review)?

Answer: N/A. No human subjects research was conducted; therefore, IRB approval is not applicable.

16. Declaration of LLM usage Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

Answer: N/A. The LLM (Qwen2.5) serves only as the audit target—it is not a component of the proposed auditing method. The auditing framework itself (min-cut computation, mutual-information lower bounds, and the dual-certificate logic) is independent of any particular language model and does not rely on LLMs as a core methodological component.

A Network Information Theory Background for Proposition 1

Conditional channel capacity. For each unlogged edge e in the time-unrolled DAG G_t, the conditional capacity c_e is the maximum information that can flow through e under the worst-case distribution consistent with the system protocol. It is not an empirical average mutual information. It is a structural upper bound read from interface specifications: token window length, hidden-dimension size, quantization level, and discrete-state cardinality. Concretely, a text channel with budget K tokens over vocabulary $\mathcal{V}$ has $c_e \le K \log |\mathcal{V}|$; a vector state of dimension d at quantization Q has $c_e \le d \log Q$; a discrete scheduler with $|\Omega|$ states has $c_e \le \log |\Omega|$. The $K \log |\mathcal{V}|$ form is the maximum-entropy bound (uniform over K-token sequences); any other distribution gives strictly lower entropy. Auditors who can measure the empirical input distribution to a particular channel may substitute a tighter bound, which only improves the certificate.

What tightening means. The static certificate remains valid under any replacement $c_e^\star \ge I(X_e; Y_e \mid \tilde{T}_t)$, so tightening is a matter of choosing smaller but still defensible per-edge budgets. In practice, this means replacing the worst-case $K \log |\mathcal{V}|$ or $d \log Q$ budget with an empirical or conditional estimate at the same audit discretization, rather than changing the theorem itself.

Cut-set upper bound and additive decomposition. Let $G_t = (V_t, E_t)$ be the time-unrolled DAG. For a cut $\Omega \subseteq V_t$ separating source nodes from a target node, the *induced cut capacity* $C_{\text{cut}}(\Omega)$ is the supremum, over joint input distributions $p(X_\Omega \mid \tilde{T}_t)$, of the directed conditional mutual information $I(X_\Omega \to Y_{\Omega^c} \mid \tilde{T}_t, X_{\Omega^c})$. The network-information-theoretic cut-set upper bound [El Gamal and Kim, 2011] states that any information flowing from sources in Ω to a target in Ω^c is bounded above by $C_{\text{cut}}(\Omega)$, and minimizing over cuts gives the tightest such bound. Proposition 1 applies this bound to the unrecorded trace fragment M_t. We use the cut-set *upper bound* direction only; the network-coding achievability results [Ahlswede et al., 2000], which concern lower bounds on capacity under cooperative coding across the cut, are not invoked. The induced capacity $C_{\text{cut}}(\Omega)$ is in general a supremum over joint input distributions and cannot be read directly from interface specifications. Lemma 1 shows that under software orthogonality, when the channel laws across the cut factor into independent single-edge channels conditional on $\tilde{T}_t$, the induced capacity is bounded above by the sum of per-edge capacities:

$$C_{\text{cut}}(\Omega) \;\le \sum_{e \in E(\Omega, \Omega^c)} c_e.$$

Corollary 1 combines this with Proposition 1 to reduce the certificate computation to a discrete minimum-cut problem on G_t with edge capacities c_e. This is the form used in the static certificate experiment and in the protocol in §B.

Formal derivation of Proposition 1. The proof in §4 has two steps: a trace-gap identity and a cut-set bound via d-separation. We restate the argument here with the d-separation step explicit.

Step 1: Trace-gap identity. Let $M_t := T_t \setminus \tilde{T}_t$. The chain rule of conditional entropy gives

$$H(S_t \mid \tilde{T}_t) = H(S_t \mid T_t) + I(S_t; M_t \mid \tilde{T}_t).$$

The first term is $\varepsilon_{\text{nominal}} = H(S_t \mid T_t)$.

Step 2: Cut-set bound via d-separation. Fix a cut $\Omega \in \text{Cuts}(U_t \to S_t)$. By the definition of a cut in the time-unrolled DAG G_t, every directed path from a node in Ω to $S_t \in \Omega^c$ must cross at least one edge in $E(\Omega, \Omega^c)$. The causal influence of M_t on S_t must be mediated by the cross-cut message stream $(X_\Omega \to Y_{\Omega^c})$ together with the boundary state X_{Ω^c}. Formally, M_t is d-separated from S_t by $(X_\Omega \to Y_{\Omega^c},\, X_{\Omega^c})$ conditional on $\tilde{T}_t$, which yields

$$I(S_t; M_t \mid \tilde{T}_t) \;\le\; I\Big(X_\Omega \to Y_{\Omega^c} \;\Big|\; \tilde{T}_t,\, X_{\Omega^c}\Big).$$

By the definition of $C_{\text{cut}}(\Omega)$ as the supremum of the right-hand side over admissible joint input distributions on X_Ω,

$$I(S_t; M_t \mid \tilde{T}_t) \;\le\; C_{\text{cut}}(\Omega).$$

Since this holds for every cut, it holds for the minimum,

$$I(S_t; M_t \mid \tilde{T}_t) \;\le\; \min_{\Omega \in \mathrm{Cuts}(U_t \to S_t)} C_{\text{cut}}(\Omega).$$

Combining with Step 1 completes the proof of Proposition 1. Corollary 1 follows immediately by substituting Lemma 1 into each software-orthogonal cut: under orthogonality, $C_{\text{cut}}(\Omega) \le \sum_{e \in E(\Omega, \Omega^c)} c_e$, and the minimum over cuts becomes a discrete min-cut problem on the unlogged subgraph with edge capacities c_e.

B Experimental Protocols

B.1 Static Certificate Computation Protocol

Inputs: deployment architecture specification, logging manifest, and protocol budgets (token window sizes, hidden dimensions, quantization levels). Procedure: (1) construct the time-unrolled DAG G_t; (2) partition edges into logged and unlogged; (3) verify software orthogonality (Lemma 1) or add shared-state edges explicitly to G_t until orthogonality holds; (4) assign capacity c_e per unlogged edge using the forms in §4; (5) enumerate reachable cut sets $\mathcal{C}_{\text{unlogged}}$ with a min-cut algorithm after logged edges are collapsed to zero residual capacity; (6) compute $\varepsilon^{\text{UB}}_{\text{state}} = \varepsilon_{\text{nominal}} + \min_C \sum_{e \in C} c_e$ via Corollary 1.

For the logging ablation, repeat steps (2)–(6) for the ordered manifests used in §6.1: output-only trace, tool-call trace, router-logit trace, summary-buffer trace, and full scratchpad trace. Report the upper-bound value and active minimum-cut edges at each level. See `experiments/7.1_static_certificate/README.md` for implementation.

B.2 Dynamic Certificate Estimation Protocol

Proxy certificate. Inputs: model, task environment, and probe family $Z_t^{(k)} = f_k(S_t)$. Steps: (1) run inference on N trajectories and record $(\tilde{T}_t, Z_t^{(k)}, A_t)$; (2) train cross-fitted predictors $p_\theta(A_t \mid \tilde{T}_t)$ and $p_\phi(A_t \mid \tilde{T}_t, Z_t^{(k)})$; (3) compute the cross-entropy difference $\hat{I}_k = L_{\text{trace}} - L_{\text{trace}+Z^{(k)}}$; (4) estimate trajectory-block bootstrap intervals; (5) repeat for random, label-permuted, 1-dimensional, 3-dimensional, 5-dimensional, and full-slice proxies.

Optional controlled replay certificate. Inputs: model and two controlled execution regimes (wild and replay). Steps: (1) for each $\tilde{T}_t$, run the same system under wild and replayed missing-state conditions; (2) estimate $\hat{\Delta} = \hat{\mathrm{JS}}(P_{\text{wild}}, P_{\text{replay}} \mid \tilde{T}_t)$; (3) aggregate over trace slices. This is not output-only paraphrase replay; it is valid only under the controlled replay condition in §5.

Intervention certificate. Inputs: model, targeted hidden module, perturbation budget, and task split. Steps: (1) inject ξ_{hidden} at levels $\{0, \sigma_1, \sigma_2, \ldots\}$ while holding $\tilde{T}_t$ fixed; (2) record $(\xi_{\text{hidden}}, A_t \mid \tilde{T}_t)$ pairs; (3) estimate $\hat{I}(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t)$ via conditional action-distribution shift for discrete action spaces, or via InfoNCE/MINE for continuous action spaces; (4) repeat on dormant tasks and active tasks under the same topology to test whether $\delta^{\text{LB}}_{\text{act}}$ changes while $\varepsilon^{\text{UB}}_{\text{state}}$ remains fixed.

Multi-agent private-report certificate. Inputs: task pool, worker model, controller model, and a two-class API space. Steps: (1) sample $N/2$ arithmetic tasks and $N/2$ factual-retrieval tasks; (2) have the worker generate a private evidence report under a prompt that forbids direct API names and audit each report for leakage; (3) sample the controller action distribution k times under wild, neutral replay, opposite-class report, same-class shuffled report, and oracle-tag conditions; (4) parse actions into {`calculator`, `search`, `invalid`}; (5) compute per-task JS divergence in bits and task-block bootstrap intervals. The oracle-tag contrast is reported only as an upper bound; the main contrast is the opposite-class worker-report intervention.

See the README files for experiments 7.2, 7.3, and 7.6 for implementation.

C Proxy Estimator: Bias and Variance

The read-only proxy experiment is a calibration of observational access, not a main activation result. On 3,000 tool-selection trajectories (mixed calculator/search/email/calendar/weather), we record $\tilde{T}_t$ (embedding-layer representation), Z_t (layer-24 tool-logit projection), and A_t. The CE-diff $\hat{I} = \mathcal{L}(A_t \mid \tilde{T}_t) - \mathcal{L}(A_t \mid \tilde{T}_t, Z_t)$ is estimated via 5-fold cross-fitted logistic regression with trajectory-block bootstrap. Table 5 reports the resolution ablation: random, 1/3/5-dim PCA, and full 10-dim proxy.

The d=5 PCA proxy recovers 0.0317 bits (1.4% of the $\log_2 5 = 2.32$ bit ceiling) with CI excluding zero on the mixed-task distribution. The magnitude reflects an *observational bottleneck*: when the baseline predictor achieves near-perfect accuracy, the ceiling $H(A_t \mid \tilde{T}_t) \approx 0$ caps any observational proxy. This is consistent with Pearl's Causal Hierarchy [Pearl, 2009]: proxy certificates operate at Level 1 (association), while the intervention and replay certificates in §6.2 operate at Level 2 (intervention), breaking the deterministic data-generating process.

The dormant/active proxy split should be read as an estimator-noise diagnostic. On calculator tasks (dormant scratchpad), $H(A_t \mid \tilde{T}_t) = 0$ empirically, yielding $\delta^{\text{LB}}_{\text{act}} = 0$ trivially. On planning tasks (active scratchpad), the proxy row is positive in point estimate but has a wide CI that straddles zero. We therefore do not use the read-only proxy as the main body evidence for active scratchpad use; the causal intervention and controlled replay in §6.2 supply that evidence.

Proxy	CE-diff (bits; nats)	95% CI (bits)
Resolution ablation (mixed task)		
random	-0.0007; -0.0005	$[-0.0012, -0.0003]$
$d = 1$ PCA	$+0.0065$; $+0.0045$	$[+0.0053, +0.0078]$
$d = 3$ PCA	$+0.0250$; $+0.0173$	$[+0.0225, +0.0273]$
$d = 5$ PCA	$+0.0317$; $+0.0220$	$[+0.0273, +0.0358]$
full (10-dim)	$+0.0183$; $+0.0127$	$[+0.0134, +0.0229]$
Dormant/active split (d=64 PCA, L2-regularized)		
calculator (dormant)	0.0000; 0.0000	$[0, 0]$
planning (active)	$+0.0412$; $+0.0286$	$[-0.0725, +0.1386]$

Table 5: Read-only proxy calibration. Top: resolution ablation on the mixed tool-selection dataset. Bottom: dormant/active split. The active-task row is reported honestly as an estimator-noise diagnostic because its confidence interval straddles zero; causal replay/intervention supplies the body evidence for active scratchpad use. Negative random-control CE-diff is a finite-sample artifact near the noise floor and is clipped to zero when interpreted as a lower-bound certificate.

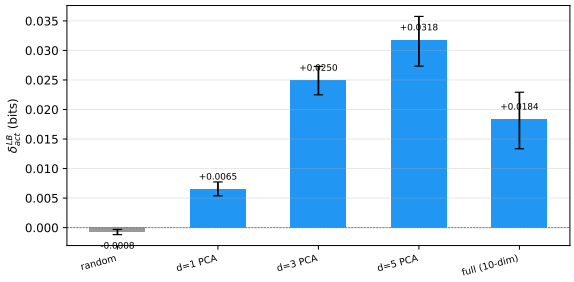

Figure 3: Proxy resolution ablation. $\delta^{\text{LB}}_{\text{act}}$ increases monotonically through d=5 PCA. The random proxy sits near zero (estimator leakage check). Error bars are 95% trajectory-block bootstrap CIs.

The CE-diff estimator $\hat{I} = L(\tilde{T}_t) - L(\tilde{T}_t, Z_t)$ lower-bounds $I(Z_t; A_t \mid \tilde{T}_t)$, but three bias sources and two variance sources matter in practice.

Bias sources. (1) *Proxy coarsening:* $Z_t = f(S_t)$ discards information in S_t not captured by f, so $I(Z_t; A_t \mid \tilde{T}_t) \leq I(S_t; A_t \mid \tilde{T}_t)$. The gap is the coarsening loss. (2) *Restricted predictor class:*

Logistic regression or shallow MLPs may not reach the Bayes-optimal predictor, so the CE-diff underestimates the true conditional MI. (3) *Finite-sample bias:* With finite N, the cross-entropy on held-out folds is an upward-biased estimate of the population CE, and the difference of two such estimates carries residual bias. Cross-fitting mitigates but does not eliminate this.

Variance sources. (1) *Trajectory sampling:* The N observed trajectories are a finite sample from the deployment distribution. Block bootstrap over trajectories estimates the resulting CI. (2) *Proxy dimension:* Higher-dimensional proxies carry more information but increase predictor variance. The PCA ablation in Table 5 shows this trade-off: d=5 outperforms both d=3 and the full 10-dimensional proxy.

Mean squared error. Decomposing MSE $=$ (bias)^2 + variance, the proxy certificate is a conservative lower bound: positive bias (underestimation) is safe for the audit (it never overstates δ_{act}), while variance widens the CI but does not shift the point estimate.

Sampling noise and temperature. Increasing decoding temperature can raise $H(A_t \mid \tilde{T}_t)$, but independent sampler noise is not a certificate amplifier. It changes the audited policy and can reduce $I(Z_t; A_t \mid \tilde{T}_t)$ when the added action variation is unrelated to the hidden state proxy. A temperature sweep is therefore a calibration of a randomized deployment policy, not a substitute for proxy quality or causal access. We do not use temperature injection for the main proxy certificate; the synthetic experiment instead injects hidden-state stochasticity with known ground truth.

D Lean 4 Formalization Boundary

The existing Lean 4 artifacts (`Screenability.lean`, `InternalImpossibility.lean`) are reinterpreted as machine-verified instances of the autoregressive zero-cut special case of Proposition 1 (Corollary 2). The file `DualCertificate.lean` formalizes structural reductions corresponding to the static-certificate cut-sum bound and the conditional-DPI dynamic certificate.

The artifact uses an axiomatic interface for measure-theoretic information theory: the structural reductions are machine-checked, while the underlying chain-rule, cut-set, and DPI facts are imported as `axioms` and listed in §D.1.

Main-Text Item	Lean File	Lean Theorem	Status
Cor 2 (Autoregressive zero-cut)	Screenability	`no_eis_autoregressive`	C
Prop 1 (General cut-sum)	DualCertificate	`prop1_static_ub`	C/E
Prop 2 (Conditional DPI)	DualCertificate	`prop2_dynamic_lb`	C/E

Table 6: C = machine-checked from Mathlib first principles. C/E = structural reduction is machine-checked conditional on the four axioms listed below. Entropy, conditional entropy, and conditional mutual information are defined by finite-discrete formulas using Mathlib finite sums and real logarithms; the four axioms in §D.1 remain external.

D.1 External / Assumed Steps

The following are imported as explicit Lean `axioms` and consumed by downstream machine-checked proofs:

- **For Prop 1:** `chain_rule` (trace-gap chain rule of conditional entropy across the visible/full-trace decomposition); `cut_set_bound` (network-information-theoretic cut-set inequality on the time-unrolled DAG).
- **For Prop 2:** `condMarkov` (conditional Markov property of probe variables); `cond_dpi` (conditional Data Processing Inequality).

The artifact verifies a structural reduction: that the main-text propositions follow from these axioms by elementary algebraic manipulation. It does not derive the axioms themselves from Lean's measure-theoretic foundations.

E Synthetic Ground-Truth Validation

To test whether the certificates recover known hidden influence, we construct a synthetic agent whose hidden-state entropy and decision relevance are controlled by design. A binary hidden variable $H \sim \text{Bern}(0.5)$ injects influence of strength β_h into the action logits; the unlogged-channel capacity $c_H = H(H) = 1$ bit is known. We run both the static min-cut ($\varepsilon^{\text{UB}}_{\text{state}} = 1$ bit) and the proxy CE-diff estimator on 1,000 trajectories at five influence levels.

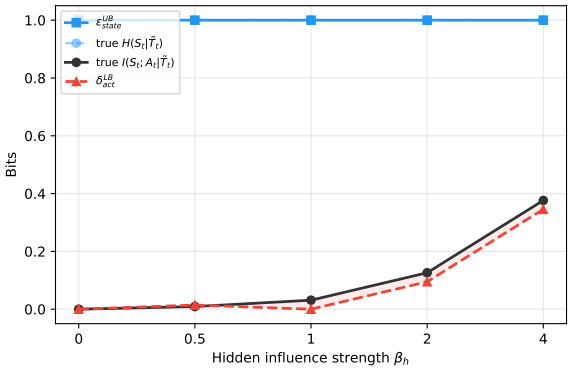

Figure 4: Synthetic ground-truth. $\varepsilon^{\text{UB}}_{\text{state}}$ (blue) equals the true $H(S_t \mid \tilde{T}_t) = 1$ bit. $\delta^{\text{LB}}_{\text{act}}$ (red) tracks the true $I(S_t; A_t \mid \tilde{T}_t)$ from below, with negative values at low β_h reflecting estimator variance near the noise floor.

Fig. 4 shows $\varepsilon^{\text{UB}}_{\text{state}} = 1$ bit matches the true hidden entropy (the bound is tight when there is no redundant unlogged capacity). The dynamic lower bound $\delta^{\text{LB}}_{\text{act}}$ tracks the true conditional MI from below at all tested influence levels ($\beta_h \geq 1$); negative CE-diff values at $\beta_h = 0$ reflect finite-sample variance around a near-zero true MI.

The Qwen deployment has a near-deterministic baseline that suppresses the proxy signal ($H(A_t \mid \tilde{T}_t) \approx 0$). The synthetic environment instead injects controlled stochasticity ($H = 1$ bit of hidden-state entropy). Without that observational bottleneck, the proxy certificate $\delta^{\text{LB}}_{\text{act}}$ tracks the true conditional-MI curve from below in the tested range. The validation shows that the proxy certificate can become tight when structural capacity is behaviorally active *and* the observational ceiling permits detection.

F Tightening the Static Certificate

The reported static certificate is intentionally conservative. Its role is to upper-bound the residual hidden-state entropy from deployment topology alone, so the default budgets in §4 are worst-case interface capacities rather than empirical information estimates.

Empirical and conditional budgets. A tighter deployment-specific certificate can replace the worst-case budget c_e on each unlogged edge by an empirically justified budget $c^\star_e$. For a text channel, one option is a high-confidence upper bound on $H(X_e \mid \tilde{T}_t)$; when the visible history strongly constrains the missing input, a conditional form such as $H(X_e \mid \tilde{T}_t, \mathcal{H}_t)$ can be smaller still. For quantized vector states, the analogous replacement is an empirical entropy or rate-distortion budget at the same audit discretization. These substitutions do not alter the min-cut proof: they only change the edge weights used by Corollary 1.

Graph refinement. If two apparent channels share latent state, naive edge-wise additivity can double-count the same information. The remedy is to refine the time-unrolled DAG so that the shared state appears as an explicit node or edge family, then recompute the min-cut on the refined graph. This usually tightens the certificate more reliably than trying to compensate with ad hoc weight corrections.

Reporting policy. For review and deployment, report a ladder of certificates: worst-case structural, empirical-marginal, and empirical-conditional. The first is the theorem-backed safety baseline. The latter two are optional tightening modes and should be reported only when the underlying measurements are defensible.